01
TUESDAY
MAY

MAY 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12 13
14 15 16 17 18 19 20 21 22 23 24 25 26 27
28 29 30 31

WK 18 (121-244)

8 am
9 am
10 am
11 am
12 noon
1 pm
2 pm
3 pm
4 pm
5 pm
6 pm

Javascript (.js)

Using the Console :-

Before coding on VS code, we'll first start code on the console direct in the browser.

Console uses REPL →

Read - Evaluate - Print - Loop
↓ reads the code
↓ calculate if req.
↓ print if req.
↓ repeat

ctrl + L → clear console

↑ → older code
↓ → newer code

What is variable :-

A variable is simply the name of a storage location. or container which stores data.

e.g. in memory storage →

variables → a [10] age [22] name [Rishabh]] containers / memory loc" / block of memory.

2018

2018 JUNE
M T W T F S S M T W T F S S
1 2 3
4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20 21 22 23 24
25 26 27 28 29 30

WK 18 (122-243)

02
WEDNESDAY
MAY

8 am
9 am
10 am
11 am
12 noon
1 pm
2 pm
3 pm
4 pm
5 pm
6 pm

Data Types in JS :-

Primitive types →

- Number
- BigInt
- Boolean
- Symbol
- String
- Undefined
- Null

→ typeof variable_name → will print type of the variable.
or typeof constant → " " " "
→ " " " constant
→ (JS automatically detects type of variable)

Numbers in JS :-

- Positive (14) & Negative (-4)
- Integers (45, -30)
- Floating numbers - with decimal (4.6, -8.9)
  ↳ other languages may have diff. data type for float

2018

JUN
JUL
AUG

Convert float to int:- n = Math.floor(5.12); → 5 (does not change original)
n = Math.floor(-2.12); → -2

03 THURSDAY MAY

Convert String to int:- parseInt("10.42") → 10
" " to number :- Number("10.42") → 10.42

8 am
9 am
10 am
11 am
12 noon
1 pm
2 pm
3 pm
4 pm
5 pm
6 pm

Number has a limit. ∴
- ~~if large float~~
- if we try to store large float then it will only store limited float precision value or round off to the nearest
- if we try to store large int value then it will return $-\infty$ or $+\infty$ (infinity)

Operators in JS:-

a = 20
b = 10

a + b → operator
operands

- addition (+)
  a + b
- Subtraction (-)
  a - b
- Multiplication (*)
  a * b
- Division (/)
  a / b
- modulo (remainder operator) (%)
  a % b
- Exponentiation (power operator) (**)
  a ** b

2018

Keyword set of reserved words that cannot be used as name of functions, labels, or variables as they are already part of the syntax of JavaScript.
Each keyword has its own meaning.

04 FRIDAY MAY

02|01|2018	Crockery + Rakhi	→	4000|- + 500|-
31|12|17	Ritesh Sneh	→	7800 + 600 + 1500 BD
10|01|17	Crockery	-	1000|-
08|01|17	Astha BIHAR	-	10,000|- Transfer
	Viserder	-	50,000|-
	Astha	-	30,000|- 2.K
	Mother	-	20,000|-

JUN JUL AUG

NaN in JS :-
The NaN global property is a value representing Not-A-Number.
- 0/0
- NaN - 1 or NaN + 1
- NaN * 1
- NaN + NaN

type of NaN → 'number'

Operator Precedence :-
This is the general order of solving an expression.

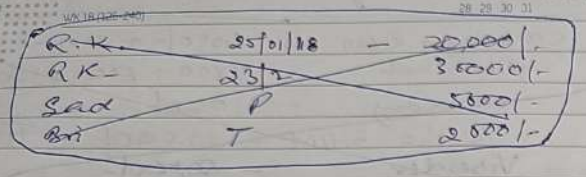
() (most)
**
* , / , %
+ , - (least)

let keyword :-
Set of declaring variables.
With this we can ~~int~~ initialize a variable with or without ~~coron~~ constant.
→ Always use let to initialize a variable (Good practice) ending with (;).

2018

05
SATURDAY
MAY

MAY 2018



8 am
9 am
10 am
11 am
12 noon
1 pm
2 pm
3 pm
4 pm
5 pm
6 pm

e.g. let age = 22;
let a = 10;
let b = 20;
let c = a+b;
let d;

Const keyword :-
values of constants can't be changed with re-assignment & they can't be re-declared.

e.g. const year = 2023;
year = 2026] Error
year = year + 1;

var Keyword :-
Old syntax of writing variables.
(We won't use)
(Always prefer let)

e.g var a = 10;

06 SUNDAY

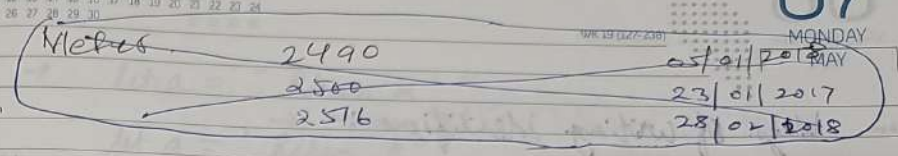
→ Already declared variable can't be declared again in same block/scope.

→ Comments starts with → //

2018

2018 JUNE

07
MONDAY
MAY



8 am
9 am
10 am
11 am
12 noon
1 pm
2 pm
3 pm
4 pm
5 pm
6 pm

Assignment Operators :-
=, +=, -=, *=, /=

Unary Operators :-
++ , --
e.g. a++;
b--;

Pre-increment (change, then use)
++a;
Pre-decrement (change, then use)
--a;
Post increment (use, then change)
a++;
Post decrement (use, then change)
a--;

Identifier Rules :-
All Javascript variables must be identified with unique names (identifiers).

* Same rules as all other programming language
e.g. can start with $ / _ / any letter. (no space)

JUN
JUL
AUG

2018

08
TUESDAY
MAY

MAY 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12 13
14 15 16 17 18 19 20 21 22 23 24 25 26 27
28 29 30 31

WK 19 (128-237)

Ways of writing identifiers :-

camelCase (JS naming convention - most used)

snake_case

PascalCase

Boolean in JS :- true/false

e.g. let isAdult = true;

→ In javascript type / datatype of a variable can be changed unlike other programming languages. Hence JS is dynamic typed.

→ What is TypeScript ? :-
TypeScript is Static Typed unlike JS which is dynamic Typed
It is designed by Microsoft.

String in JS :- (primitive data type)
Strings are text or sequence of characters.
Both double quotes "" and single quotes '' are valid in JS.
e.g.
let name = "Ritank";
Or let name = 'Ritank';

let empty = "";

For convention, use '' for single letters, use "" for multiple.

2018

2018 JUNE
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20 21 22 23 24
25 26 27 28 29 30

WK 19 (129-236)

09
WEDNESDAY
MAY

→ let a = "Ritan 'k' ";
let a = 'Ritan "k" '
let a = 'Ritan 'k' ' → error
let a = "Ritan "k" " → error
let a = "Ritan "k" "

→ String Indices / Index :- (0 base indexing)

let name = "Ritank";
0 1 2 3 4 5

name[0] → 'R'
name[4] → 'n'
name[1000] → undefined

name.length → 6

name[name.length - 1] → 'k'

Weird → "Ritank".length → 6
"Ritank"[0] → 'R'

Concatenation :-
Adding strings together

"Ritank" + " " + "Jaikar" → "Ritank Jaikar"

"Ritank" + 1 → "Ritank1"
string + num = string

2018

JUN
JUL
AUG

10 THURSDAY MAY

WK 19 (130-235)

MAY 2018

M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12 13
14 15 16 17 18 19 20 21 22 23 24 25 26 27
28 29 30 31

8 am

null & undefined in JS :-

9 am

undefined
A variable that has not been assigned a value is of type undefined.
e.g. let a; → undefined
typeof a → undefined (in JS → may be nhi pata)

12 noon

null
The null value represents the intentional absence of any object value.
To be explicitly assigned.
e.g. let a = null;
a → null

1 pm

console.log() :-

To write (log) a message on the console

e.g.
```
console.log("Hello");   → Hello
console.log(123);       → 123
console.log(1+2);       → 3
let a = 10;
console.log(a);         → 10
console.log("Hello", a, "1+2");
                        Hello 10 3
```

Linking JS File :-

```
<script src="app.js"></script>
```
↳ Can be added in head. But to load HTML & CSS first, it is added before </body>. (Good practice)

2018

JS Unlike console in JS file lines should be ended with ';'.
In console ';' is not mandatory, but in JS file it is mandatory.

2018 JUNE

M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20 21 22 23 24
25 26 27 28 29 30

WK 19 (131-234)

11 FRIDAY MAY

8 am

Template Literals :- (for better exp.)

(or String Interpolation)

They are used to add embedded expressions in a string.

9 am

e.g.
```
let a = 5;
let b = 10;
```

Normal way ↓ or ↱ let price = "Price is" + (a+b) + "rupees";
console.log(price);

10 am

```
console.log("Price is", a+b, "rupees");
```

11 am

Better way →

```
console.log(`Price is ${a+b} rupees`);
```

Back ticks — *embedded expression.

12 noon

Syntax → ` (string) ${ (number/calculation} string `

1 pm

Operators in JS :- (all)

2 pm

- Arithematic - (+, −, *, /, %, **)
- Unary - (++, −−)
- Assignment - (=, −=, +=, /=, %=, *=, etc.)
- Comparison - (>, <, >=, <=, ==, !=, ===, !==)
- Logical - (&&, ||, !)
- Conditional (Ternary) - ((condition) ? (true) : (false))

3 pm

4 pm

5 pm

6 pm

(more)

2018

JUN JUL AUG

12 SATURDAY MAY

MAY 2018

W/K 19 (132-233)

8 am

Comparison Operator (===, !==)

9 am

The other comparison operators compares value not type.

10 am

But (===, !==) compares type & value.

11 am

==

e.g. "122" == "122"
→ true

• 1 == '1'
→ true

• 0 == ''
→ true

• 0 == false
→ true

12 noon

1 pm

2 pm

3 pm

4 pm

5 pm

6 pm

• null == undefined
→ true

===

e.g. "122" === 122
→ false (diff type)

• ''
→ false

• ''
→ false

• ''
false

• ''
→ false

Comparison for non-numbers :-

13 SUNDAY

e.g. 'a' > 'A' → true
'a' > 'b' → false
'*' < '0' → false

This is bec every character is identified or assigned with a Unicode (Just like ASCII in C)

2018

2018 JUNE

W/K 20 (134-231)

14 MONDAY MAY

e.g. 'a' → 61, 'b' → 62
'A' → 41, 'B' → 42

Hence, 'a' < 'b' < 'c' < 'd' ...
'A' < 'B' < 'C' < 'D' ...
'a' > 'A'

Conditional Statements :-

- if else
- nested if else
- Switch

if Statement :-

e.g. let a = 22;
if (age >= 18) {
console.log("You can vote");
}
else {
console.log("You can not vote");
}

(if else if, nested if else → same as C)

Logical Operators :-

Logical operations to combine expressions (combining multiple conditions)

Logical AND → &&
Logical OR → ||
Logical NOT → !

2018

JUN JUL AUG

→ Convert string to int : parseInt ('123'); → 123
or parseInt ('123abc'); → 123
or parseInt ('abc123'); → NaN
or parseInt ('1.9'); → 1 (Int part only)

15 TUESDAY MAY

MAY 2018
M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12 13
14 15 16 17 18 19 20 21 22 23 24 25 26 27
28 29 30 31

WK 20 (135-230)

8 am
9 am
10 am
11 am
12 noon
1 pm
2 pm
3 pm
4 pm
5 pm
6 pm

truthy & falsy :-

Everything in JS is true or false. (in boolean context)

This doesn't mean their value is false or true, but they are treated as false or true if taken in boolean content.

Falsy values :-
false, 0, -0, 0n (bigint value), "" empty string, null, undefined, NaN.

Truthy values :-
everything else.

Switch Statement :- (same as C)

Alert & Prompt :-

- Alert displays an alert message on the page.

e.g alert ("Something is wrong!");

- Prompt displays a dialog box that asks user for some input and returns string.

e.g prompt ("please enter your name");

↳ we can also put entered value in a variable
e.g let name = prompt ("");
(only returns string)

2018

→ Convert string to number : Number ('10.123') → 10.123
another way using unary operator +
e.g. let a = +"100.24" → 100.24

MAY 2018
M T W T F S S
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20 21 22 23 24
25 26 27 28 29 30

WK 20 (136-229)

16 WEDNESDAY MAY

→ error :-
Console.error ("This is an error msg");
↳ will display on the console (error message)

→ warning :-
Console.warn ("This is a warning msg");
↳ will display on the console (warning message)

String Methods :-
Methods - actions that can be performed on Objects.

Format : stringName.method()

e.g. console.log()

- str.trim()
Trims whitespaces from both ends of string & returns a new one. (It applies on the ends only)

let msg = " Hello ";
msg.trim();
↳ 'Hello'

→ Strings are immutable in JS
No change can be made to string.
Whenever we do try to make a change, a new string is created and old one remains same.
By applying any method it does not change original string.

2018

JUN JUL AUG

17 THURSDAY MAY

WK 20 (137-228)

MAY 2018

M	T	W	T	F	S	S
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

8 am

- str.toUpperCase();

9 am

- str.toLowerCase();

10 am

→ String Methods with Arguments :-
Argument is a some value that we pass to the method.
Format : stringName.method(arg);

11 am

- indexOf
Returns the first index of occurence of some value in string. Or gives (-1) if not found.

12 noon

e.g. let str = "Ritank"
str.indexOf(";") → 1
str.indexOf("tank") → 2
str.indexOf("J") → -1
str.indexOf("rit") → -1

1 pm

2 pm

3 pm

4 pm

5 pm

6 pm

→ Method Chaining
Using one method after another. Order of execution will be left to right.
e.g. str.toUpperCase().trim();
↳ multiple methods can be used

- slice
Returns a part of the original string as new string.
e.g. let str = "Ritank Jaiker";

2018

2018 JUNE

M	T	W	T	F	S	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	

WK 20 (138-227)

18 FRIDAY MAY

8 am

- str.slice(2); → tank Jaiker
↳ from 2nd index to last

9 am

- str.slice(2, 6); → tank

10 am

start index (inclusive) end index (non-inclusive)

11 am

- str.slice(-num) = str.slice(length - num)
str.slice(-2); → er

12 noon

(substr() is almost same as slice())

1 pm

2 pm

- replace
Searches a value in the string & returns a new string with the value replaced. It only replaces first occurance

3 pm

let str = "I love coding";

4 pm

str.slice(1);
str.replace("love", "do") → I do coding

5 pm

str.replace("I", "") → love coding

6 pm

str.replace("I", "We") → We love coding

- repeat
Returns a string with the numer of copies of a string.
e.g. let str = "Mango";
str.repeat(3); → "MangoMangoMango"

JUN JUL AUG

2018

Arrays are special type of objects in Javascript. (Indexes are keys here)

19 SATURDAY MAY

MAY 2018

wk 20 (139-226)

Array (Data Structure) :-
Linear collection of data (can be of similar types or different types) (unlike other P.L.)

e.g. let marks = [99, 95, 80, 91, 72]
let name = ["aman", "ritik", "Ron"];
let info = ["Ron", 4, 10, 9];

// empty array
let newArr = [];
newArr.length; → 0
or [].length → 0
[1,2,3,4].length → 4
marks[0] → 99
name[0] → aman
name[0].length → 4
name.length → 3
name[0][1] → m

Arrays are Mutable :-
e.g. let num = [1, 2, 3];
num[0] = 4;
now num → 4, 2, 3
num[10] = 6;
num.length → 11
num → 4, 2, 3, 10
index 3-9 are empty

So array length will be considered until last element, no matter if array is empty in between.

20 SUNDAY

2018

2018 JUNE

wk 21 (141-224)

21 MONDAY MAY

Array Methods :-

- Push : add to end
  e.g. num.push(20); name.push("Ritank");
- Pop : deletes last element & returns it
  e.g. num.pop();
- Unshift : add to start
  e.g. ~~num.push(5)~~ num.unshift(5);
- Shift : deletes first element and returns it
  e.g. num.shift();
- indexOf : returns index of something, if not found the (-1)
  e.g. num.indexOf(1);
- includes: search for a value (true/false)
  e.g. name.includes("Ritank");
- concat : merge 2 arrays (we concatenate) (does not change original)
  e.g. num.concat(name); → name array is added in num.
- reverse : reverse an array (reverse original)
  e.g. name.reverse();
- slice : copies a portion of an array (does not change original)
  e.g. name.slice() → original array (other syntax is same as string slice)
- Always pass valid parameters in slice, other wise it will return empty array.

JUN JUL AUG

2018

• reduce : arr.reduce(callbackFn, initialValue); → (optional)
e.g. arr.reduce((accumulator, currentValue) => accumulator + currentValue, 0);
(Sum) ↳ of elements in array

22 TUESDAY MAY
WK 21 (142-223)

(changes the original)

8 am • splice : removes / replace / add elements in the place.

9 am Format : arrayName.splice(start, deleteCount, item0, ..., itemN)

10 am e.g. colors.splice(4); → will return array from 4th index to end and removes it from original.

11 am colors.splice(0, 2) → will return array from 0th index until count of 2. and deletes it from original

12 noon

1 pm colors.splice(1, 3, "red", "black"); → will return 3 items from 1st index and deletes it, and add "red", "black" in their place.

2 pm

3 pm colors.splice(0, 0, "red", "black"); → will return empty array, and does not delete any thing and add "red", "black" in starting.

4 pm

5 pm

6 pm • sort : sorts in array (default : ascending order)
e.g. let char = ['b', 'r', 'k', 'a', 's'];
char.sort(); → ['a', 'b', 'k', 'r', 's']

let days = ["monday", "tuesday", "sunday"]
char.sort(); → ["monday", "sunday", "tuesday"]

sort method sorts the strings on the basis of unicode of first letter or 0th index, if unicode is same or same letter then move to next letter/index.

2018

Array to String : arr.toString(); → converted array to string separated by commas
e.g. [1, 2, 'Ri', 3, '#'] → "1, 2, Ri, 3, #"

2018 JUNE

23 WEDNESDAY MAY
WK 21 (143-222)

let num = [100, 24, 39, 17, 300]
num.sort(); → [100, 17, 24, 300, 39]

sort even converts numbers to their unicode, and works same as string, hence sort may not work for numbers.

Array References :- (address in memory)

[1] === [1] → false
[1] === [1] → false (since to diff. references)

If we compare two array or array names then it will compare address of the arrays not the values.

let arr = ['a', 'b'];
let arrCpy = arr; → now arr also has same or point to same address
Hence if we change arrCpy then arr will also change.

e.g. arrCpy.push('c'); → arrCpy → 'a', 'b', 'c'
also, arr → 'a', 'b', 'c'
arr === arrCpy → True

Constant Arrays :-
const arr = [1, 2, 3, 4];
this means arr has constant address, we can't change the address of arr, means we can't assign another array to arr.

Element can be altered (like, push, pop)

2018

JUN JUL AUG

24 THURSDAY MAY

WK 21 (144-221)

MAY 2018

Nested Arrays :- (multidimensional arrays)
(arrays of arrays)

e.g. let nums = [[1,2,3,4], [1,2,3], [1]]
↳ arr size 4 ↳ arr size 3 ↳ arr size 1

unlike other programming languages different columns can be assigned to different rows.

for loop :- (same as C)
e.g. for(let i=1; i<=5; i++) {
console.log(i);
}

while loop :- (same as C)
e.g. let i=1;
while (i<=5) {
console.log(i);
i++
}

for of loop :- (like for each loop in C)
format: for (let element of collection) {
// code
}
e.g. let fruits = ["mango", "apple", "litchi"];
for (let fruit of fruits) {
console.log(fruit);
}

2018

2018 JUNE

25 FRIDAY MAY

WK 21 (145-220)

e.g. for (let char of "Ritank") {
console.log(char);
}
→ R i t a n k

JS Object Literals :- (data structure)
(it's like structure in C)
Used to store keyed collections & complex entities.
(User defined objects)
Object Literals are different from Object in JS.
Objects are much broader word in JS.
Object Literals can be said to be a Object but not vice versa.

Property => (Key, value) pair

Objects are collection of properties.

- JS object literal is a data type used to define objects in JavaScript. It is a syntax for creating key-value pairs inside curly braces with semi-colon at the end. and all pairs seperated by commas.

e.g. (Syntax)

```
let/const student = {
    name : "Ritank",
    age : 22,
    marks : 86%,
    city : "Gurgaon"
};
```

using const reference/address of student is fixed.
Hence, const will be used for object literals.

JUN JUL AUG

obj_name.key → do not work for numbers as key and variables. (Drawback)

26 SATURDAY MAY — WK 21 (146-219)

MAY 2018

8 am Get values –

Syntax: obj_name["key"]; (Universal)
or obj_name.key; (better) easy

9 am e.g. student["name"]; → Ritank

10 am student.age; → 22

11 am e.g.

12 noon ~~obj~~ let prop = "name";

1 pm student.prop; ✗ (~~var~~ since prop is a variable)

2 pm student[prop]; ✓ → Ritank.

(key as a variable)

3 pm

- numbers can also be assigned as key
- JS automatically converts objects keys to strings
- Even if we made the number as a key, the number will be converted to string.
- keyword can ~~be~~ also be named as keys, but stores as a string, but not good practice.

4 pm

5 pm

6 pm

e.g. const obj = {
1: 'a',
2: 'b',
true: 'c',
null: 'd',
undefined: 'e',
};

27 SUNDAY

2018

obj_name["key"] → " " is not mandatory for numbers / variables / true / false / null / undefined / etc. as a key. ~~Here will automatic obj~~

2018 JUNE

28 MONDAY MAY — WK 22 (148-217)

8 am obj[1] → 'a'
obj[true] → 'c'
obj[null] → 'd'
9 am obj[undefined] → 'e'

', true, null ... all are first converted to string.

10 am obj.1 ✗ it does not convert number to string.
obj.null ✓ → 'd'
11 am obj.true ✓ → 'c'

12 noon • Generally keys are simple word like variable.

1 pm Add/Upate value :-

student.name = "Aman";

2 pm student["age"] = 30;

3 pm add new key-value pair ↓

4 pm student.gender = "male";
↳ new key is added

5 pm

6 pm • value can be converted to string, number, arrays etc.

~~delete~~ delete key ↓

delete student.gender;
↳ 'deleted.

(delete is not ~~used~~ much used)

JUN JUL AUG

2018

29 TUESDAY MAY

MAY 2018

WK 22 (149-216)

Object of Objects :-

e.g. Storing information of multiple students :

```
const classInfo = {
    aman : {
        grade : "A"
        city : "Delhi"
    },
    raman : {
        grade : "A+"
        city : "Pune"
    },
    pavan : {
        grade : "O"
        city : "Banglore"
    }
};
```

accessing individual value :-

e.g.

classInfo.pavan.city ; → "Banglore"

Array of Objects :-

e.g.

```
const classInfo = [
    {
        name : "Aman"
        city : "Delhi"
    },
    {
        name : "raman"
        city : "Pune"
    },
    {
        name : "pavan"
        city : "Banglore"
    }
]
```

accessing individual element :-

e.g. classInfo[1].name ; → "raman"

↳ index

Math Object :-

- Type Math on console for all operations

e.g. Math.PI → 3.1415...
Math.E → 2.7182...

} Properties

Methods :-

e.g.

Math.abs(-12) ; → 12

Math.pow(2,4) ; → $2^4 = 16$ (same as **)

Math.floor(5.9) → 5 (round off to smaller int number)
Math.floor(-4.8) → -5

Math.ceil(5.01) → 6 (round off to nearest larger int number)
Math.ceil(-5.5) → -5

2018 JUNE

30 WEDNESDAY MAY

WK 22 (150-215)

JUN JUL AUG

2018

31
THURSDAY
MAY

W/K 22 (151-214)

MAY 2018

M	T	W	T	F	S	S	M	T	W	T	F	S	S
	1	2	3	4	5	6	7	8	9	10	11	12	13
14	15	16	17	18	19	20	21	22	23	24	25	26	27
28	29	30	31										

8 am Math.random() → random value [0, 1)
from 0 to 1, 0 is inclusive
1 is exclusive.

9 am

10 am Random Integers for fixed Range :-

11 am e.g. 1 to 10 ↴

12 noon let num = Math.random();

1 pm num = num * 10;

2 pm num = Math.floor(num);

3 pm num = num + 1;

e.g.
→ 0.674...
→ 6.74...
→ 6 → (0, to 9)
→ 7 → (1 to 10)

4 pm in 1 line: ↴

5 pm let num = Math.floor(Math.random() * 10) + 1;

6 pm Formula for generating (random) integers in range of [min, max]

⇒ Rnum = Math.floor(Math.random() * (max - min + 1)) + min;

2018

JULY 2018

M	T	W	T	F	S	S
30	31					1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29

MON	TUE	WED	THU
4	5	6	
11	12	13	
18	19	20	
25	26	27	

01
FRIDAY
JUNE

JUNE 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20 21 22 23 24
25 26 27 28 29 30

WK 22 (152-213)

8 am Functions in JS :- (similar to C)
A function in programming is a block of code which
9 am can be ~~be~~ called any time by writing function
name and passing arguments/values in curved ~~&~~ brackets
10 am if required. parameters / values

11 am → Function Definition

12 noon
```
function funcName (empty/~~argu~~ Parameters ...) {
    // code
}
```
1 pm

→ Function calling

2 pm
```
funcName (empty / arguments ...);
```
3 pm

Return :-
4 pm return keyword is used ~~to~~ to return value from function.
A function can only return one value at a time.
5 pm After return, function ends, if there is any code
after return, then it won't execute.
6 pm

Scope :-
Scope determine ~~the~~ accessibility of variables, objects,
and functions ~~from~~ ~~a~~ different parts.

- Function - (inside function)
- Block - (inside { } - applied for let, const not var (older))
- Lexical - (A ~~any~~ variable defined ~~in~~ outside func. can be accessible inside another func. ~~def~~ (nested func.) defined after variable declaration. ~~The~~ But not vice-versa.
- Global - (not inside any)

2018

2018 JULY
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16 17 18 19 20 21 22
23 24 25 26 27 28 29 30 31

WK 22 (153-212)

02
SATURDAY
JUNE

→ Nested function in JS is possible.

8 am
Function Expressions :-
9 am Directly storing function in a variable without
naming function.
10 am e.g.
```
const sum = function (a, b) {
                return a + b;
            }
sum (2, 3);  → 5
```
11 am

12 noon
1 pm Higher Order Functions :-
A function that does ~~both~~ one or both :
2 pm (callbacks) • takes one or multiple functions as ~~arg~~ parameter or arguments
when func. call.
• returns a function.
3 pm

e.g.
```
function multipleGreet (func, n) {
    for (let i=1; i<=n; i++) {
        func()
    }
}
let greet = function () {
        console.log ("hello");
};
multipleGreet (greet, 4);  → 4 times "Hello"
```
4 pm
5 pm
6 pm

SUNDAY 03

e.g. (returns a function) - ~~Eg~~ (Works as ~~a~~ a factory function)
```
function oddEvenTest (request) {
    if (request == "odd") {
        return function (n) {
            console.log (!(n % 2 == 0));
        }
    }                    // checks if odd or not
```

JUL
AUG

2018

Callbacks :- A callback is a function passed as an argument to another function. (Remember not to use parenthesis)
↳ Otherwise it will take as function call and execute the function immediately.

04 MONDAY JUNE

WK 23 (155-210)

```
else if (request == "even") {
        return function (n) {
                console.log (n % 2 == = 0);
        }
}       // checks if even or not.
else {
        console.log ("Wrong request");
}
}
```

let func = oddEvenTest ("odd"); // now func will check if no. is odd or even.
func (10); → False (not odd)

Methods :- (functions inside objects)
Actions that can be performed on an object.
e.g.

```
const calculator = {        → value
        add: function (a, b) {
                return a+b;
        },
        sub: function (a, b) {
                return a-b;
        }
        mul: function (a, b) {
                return a*b;
        }
};
```

calculator.add (1, 2); → 3

2018 JULY

05 TUESDAY JUNE

WK 23 (156-209)

Method (shorthand) :-
e.g.

```
const calculator = {
        add (a, b) {
                return a+b;
        }
        sub (a, b) {
                return a-b;
        }
}
```

key → add, value → return a+b;

⇒ Array & ~~Strings~~ in JS are internally objects.
(Hence, multiple built-in methods were possible)

Strings in JS are of two types
→ String primitives (datatype) - always used
→ String Objects (not in practice)

e.g. str1 = "Ritwik" → a literal is string primitive
str2 = String (1) → converted to string primitive "1"
str3 = String ("Hello") → string primitive
str4 = new String (str1) → String with new returns a string wrapper object

this keyword :-
'this' keyword refers to an object that is executing the current piece of code.
Generally, 'this' keyword is used to use object's key-value pairs as parameter inside ~~the~~ object's methods (functions), because we can not use them directly.

JUL AUG

2018

06
WEDNESDAY
JUNE

JUNE 2018

WK 23 (157-208)

Student Object

e.g. const student = {
name : "Ritank",
marks 1 : 91,
marks 2 : 92,
marks 3 : 93,
} ← Key's value.

here 'this' used to access student object's

method → getAvg() {
let avg = (this.marks1 + this.marks2 + this.marks3) / 3;
console.log(this);
return avg;
}
}

here 'this' refers to student object, hence prints whole student object.

consol.log(this);

here 'this' is not inside any self defined object, hence here 'this' refers to window object which is the main object inside which all the code/function/objects/methods exists. (inbuilt / user defined both)

Window Object :-
→ Whenever we open JavaScript / HTML file/page, a 'window' object is created which includes, all the code/everything. Can be printed on console. Type → 'window'.
Hence, if we write any windows. before inbuilt function, even then it will works. (but not required)
e.g.
window.alert("hello");

Window object is the global object (global space)

Every thing on the top (means not inside any function, etc.) is said to be in global space.

07
THURSDAY
JUNE

WK 23 (158-207)

try & catch :-
The 'try' statement allow you to define a block of code to be tested for Errors while it is being executed.
The 'catch' statement allows you to define a block of code to be executed, if an error occurs in the try block.

e.g. try {
console.log(a);
} catch {
console.log("variable a doesn't exist");
}

→ This is use to prevent website from crashing.
→ only 'try' will not execute, 'catch' is mendatory to write.
→ error can be printed also
e.g.

try {
console.log(a);
} catch (err) {
console.log(err);
}

Miscellaneous Topics

Arrow Functions :-
• Format : const func = (empty/parameters...) => {function definition}

e.g. const sum = (a, b) => {
console.log(a + b);
}

→ no 'function' keyword required.

JUL
AUG

2018

08
FRIDAY
JUNE

JUNE 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20 21 22 23 24
25 26 27 28 29 30

WK 23 (159-206)

→ for single parameter, curved brackets may be ~~~~ ignored.
e.g. const square = n => {
return n*n;
}

→ Implicit return in arrow function.
(Automatic) When function returns something without any other line of code, then return statement and ~~ curly brackets can be ignored, to make it even more compact.
e.g.
const cube = n => (n*n*n);] Both are valid
or
const cube = n => n*n*n;] syntax

Set Timeout :-
Format: setTimeout (function , timeout);
callback
time in ms (milli second)

→ setTimeout() is a global method (inside Window object), which executes the function or specified piece of code ~~ once given timer is expired.
→ Other codes or code after setTimeout() will keep on ~~ executing, and will not wait for ~~ setTimeout().
→ This is mostly used for API calls, request response, etc. (usually)
e.g.
setTimeout(() => { console.log("Hello") }, 4000);
Prints "Hello" after 4 s

2018 JULY
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16 17 18 19 20 21 22
23 24 25 26 27 28 29 30 31

WK 23 (160-205)

09
SATURDAY
JUNE

Set Interval :-
Same as setTimeout(), but it executes infinitely after given ~~ time.
Format is same as setTimeout().
→ To stop setInterval():
- Store the id.
let id1 = setInterval(() => { console.log("Hello"), 2000);
↳ prints Hello after every 2 seconds.
- To stop write: (on console)
clearInterval(id1); → stops the execution

'this' keyword with Arrow Functions :-

Normal Function	Arrow Function
Here 'this' has normal scope. means ↴ Here 'this' is ~~ parent's parent object.	Here 'this' has lexical scope. means ↴ Here 'this' is calling parent object.

e.g.
const obj = {
Key1 : this, // global scope (Window) → object
normal function → Key2 : function() {
console.log(this); // key2() scope i.e. (obj) → object
}
arrow function → Key3 : () => {
console.log(this); // parent's scope i.e. (lexical) obj's scope i.e. (window) → object.
}

(drawback, we will not be able to use 'this' to call ~~ obj using arrow function)

JUL
AUG
2018

11 MONDAY JUNE

JUNE 2018

WK 24 (162-203)

(Still there is advantage of arrow function) :-
// using setTimeout.

key 4 : function() {
Arrow function → setTimeout(() => { console.log(this); }, 2000);
(lexical) ↓ → or, function()
// parent scope i.e. key4()'s scope
i.e. (obj) → object.

key 5 : function() { ← callback
Normal function → setTimeout(function() { console.log(this); }, 2000);
↓
// setTimeout's scope i.e. Global scope
in built func. ← i.e. (window) → object.
// here 'this' is called by 'setTimeout' not 'obj'.

→ For normal function 'this' refers to the object which is calling it.
For arrow function 'this' refers to it's parent's object.

Q. Write a function that prints "Hello World" 5 times at intervals of 2s each.
let id = setInterval(() => { console.log("Hello World"); }, 2000);
setTimeout(() => { clearInterval(id); }, 10,000);
↓
calls clearInterval(id) after 10s and stops the execution of setInterval.

2018

2018 JULY

WK 24 (163-202)

12 TUESDAY JUNE

More Array Methods :- (higher order functions) (uses callbacks)

8 am

- forEach
e.g. let arr = [1,2,3,4,5];
function print(el) {
console.log(el);
}
arr.forEach(print); → 1 2 3 4 5
OR
arr.forEach(function(el) {
console.log(el);
});
OR // arrow function
arr.forEach((el) => {
console.log(el);
});

- map (very similar to forEach)
let newArr = arr.map(some function definition or name);
It maps every returned value from function to new array
e.g. let num = [1,2,3,4];
let double = num.map(function(el) { (required) {
return el * 2;
});
double → 2 4 6 8
If function does not return then:
double → [undefined, undefined, undefined, undefined]

parameters → index, array are optional

JUL AUG

2018

13 WEDNESDAY JUNE

JUNE 2018

WK 24 (164-201)

- filter

callback → true → element gets added in new array.
false → element rejected

e.g. create new array of only evens from previous array.

```
let nums = [1, 2, 3, 4, 5, 6, 7, 8];
let ans = nums.filter((el) => {
    return el % 2 == 0;
});
ans → [2, 4, 6, 8]
```

- every (similar to logical AND)

every → returns true if every element of array gives true to some function.
else returns false

arr.every(some function definition or name);

e.g. [1,2,3,4].every((el) => (el % 2 == 0));
↳ false

[2,4].every((el) => (el % 2 == 0));
↳ true.

- some (similar to logical OR)

some → returns true if any element of array gives true for some function.
else returns false

arr.some(some function definition or name);

2018

2018 JULY

WK 24 (165-200)

14 THURSDAY JUNE

- reduce

Reduces the array to a single value.

arr.reduce(reduce function with 2 variables for (accumulator, element)); → both parameters impt.

Another argument can be added in arr.reduce with initial value. (if array is empty then returns initial value)
(Accumulator starts with null if no initial value)

function for every element works as ↓

ele 1 → acc, el → returns value (stores value in not acc)

ele 2 → acc, el → returns value

ele 3 → acc, el → returns value

⋮

last ele → acc, el → ans

e.g.

```
[1,2,3,4].reduce((res, el) => (res + el));
```

↳ null + 1 → 1
1 + 2 → 3
3 + 3 → 6
6 + 4 → 10 (ans)

Find maximum in array using Reduce.

```
[1,2,3,4].reduce((max, el) => {
    if (el > max) return el;
    else return max;
});
```

↳ 4 (ans)

JUL AUG

2018

15 FRIDAY JUNE

JUNE 2018

WK 24 (166-199)

8 am Default Parameters :-
Giving default value to parameters.

9 am e.g. function func(a, b = 2){
// code
}
↳ during function call if 2nd argument is not passed then b will take value 2.

10 am

11 am

12 noon e.g function sum(a, b = 3){
return a+b;
}

1 pm sum(5); → 8

2 pm sum(5, 5); → 10

3 pm This must be used for end parameters.

4 pm Spread :-
Expands an iterable into multiple (all) values.
Iterable e.g. arrays, strings
e.g.
let arr = [1,2,3,4,5];
console.log(...arr); → 1 2 3 4 5
↳ prints individual elements

6 pm console.log(..."Ritank"); → R i t a n k

String to array → let chars = [..."Ritank"];
chars → [R, i, t, a, n, k]

Add arrays → let arr1 = [1,2,3,4]; let arr2 = [5,6,7];
(order is impt.) let arr3 = [...arr1, ...arr2]; [...arr1, ...arr2];
arr3 → [1,2,3,4,5,6,7]

2018

2018 JULY

16 SATURDAY JUNE

WK 24 (167-198)

8 am Math.min, .max :-
e.g. Math.min(1, 2, 4, 0, 3); → 0
Math.max(1, 6, 8, 4, 2); → 8

9 am Math.min(...arr3); → 1

10 am Math.max(...arr3); → 7

11 am Spread with object literals :-
e.g. let data = {
name: "Ritank",
age: 22
}

12 noon (copy object → (can be used to merge objects)) let copydata = {...data, id: 123};
↳ copys data object and add one more key-value pair id.

1 pm copy data → {name: "Ritank", age: 22, id: 123}

2 pm array to object → let obj1 = {...[1,2,3,4]};
↳ key is index of every element

3 pm obj1 → {0:1, 1:2, 2:3, 3:4}

4 pm string to object → let obj2 = {..."Ritank"};
obj2 → {0:'R', 1:'i', 2:'t', 3:'a', 4:'n', 5:'k'}

Rest :- (opposite of spread)
Allows a function to take an indefinite number of arguments/parameters and bundle them in array.

Way 1: Using arguments keyword
arguments inside function is collection of all the parameters. It is similar to array, but not array and array method won't work.

SUNDAY 17

JUL AUG

2018

18 MONDAY JUNE

JUNE 2018

WK 25 (169-196)

e.g. function func() {
console.log(arguments.length);
}
func(1, 2, 3, 4); → 4

Way 2: (better)
We can store all arguments/parameters in an array
e.g. function sum(...args) { let sum=0;
for(let i=0; i<args.length; i++) {
sum = sum + args[i];
}
return sum;
}
sum(1,2,3,4,5); → 15

...args should be at last as parameter in function definition.

Destructuring :- For array :-
Storing values of array into multiple variables.

let names = ["tony", "bruce", "steve", "xyz", "abc", "pqr"];
let [winner, runnerup, ...nameset]
let [winner, runnerup, ...others] = names
→ (rest)

winner → "tony"
runnerup → "bruce"
others → ["steve", "xyz", "abc", "pqr"]

2018

2018 JULY

WK 25 (170-195)

19 TUESDAY JUNE

For objects :-
e.g. const student = {
name: "Raj",
age: 15,
class: 10,
username: "raj@123",
password: "abcd"
}
let {username: user, password: secret = "zxcv", city: place = "Pune", age, height} = student;

user → "Raj"
secret → "abcd" (if there was no password in student the it would have taken default value "zxcv")
place → "Pune" (even there is no city, since default value is given, it will take default value)
age → 15 (age is there, and here age becomes variable)
height → X (there is no height in student)

// Basics of Javascript programming is completed
// Now starting with that part of JS which will work along with HTML, CSS and actually built websites.

JUL AUG

2018

20 WEDNESDAY JUNE

JUNE 2018

M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20 21 22 23 24
25 26 27 28 29 30

WK 25 (171-194)

8 am

DOM (Document Object Model)

9 am The DOM represents a document with a logical tree. It allows us to manipulate/change webpage content (HTML elements).

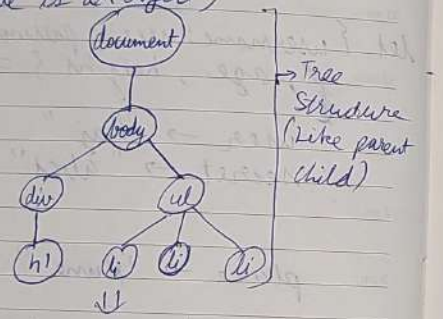
10 am

11 am e.g. ○ → node (Every node is an object)

12 noon
```
<body>
  <div>
    <h1>Todo</h1>
  </div>
  <ul>
    <li>Eat</li>
    <li>Code</li>
    <li>Sleep</li>
  </ul>
</body>
```

1 pm

2 pm ⇒



→ Tree Structure (like parent child)

3 pm

4 pm Every node is an object itself, document is the root node, which is also inside window object (Global object).

5 pm

6 pm

Console → document → (shows full code of HTML)
~~console.log(doc~~ (or window.document)
console.dir(document) → (shows all the properties/objects/methods inside document of object)

Manipulating/Changing HTML doc using JS is 2 step process:-
1. Select
2. Change/Manipulate.

2018

2018 JULY

M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16 17 18 19 20 21 22
23 24 25 26 27 28 29 30 31

WK 25 (172-193)

21 THURSDAY JUNE

console.dir() :- is a method that displays an interactive list of the ~~prob~~ properties of the specified JavaScript object.

8 am

9 am

document.all[index] :- is a collection (similar to array) of all HTML elements that are in document. The elements appear in the array in the order in which they were created.

10 am

11 am e.g. 0: html
1: head
2: meta
⋮ (and so on)

12 noon

1 pm

2 pm ~~Selecting~~ & changing innerText using document.all ↓

e.g.

3 pm
- document
  - all
    - 8: h1
      - innerText: "Spider Man"

4 pm

5 pm

6 pm

↳ change "Spider Man" to "Peter Parker".

```
document.all[8].innerText = "Peter Parker";
```

↳ This method/property is not recommended to use. ~~Instead~~ Instead of this we will use other methods. Because this method might not work in all browsers. And it is outdated also. Instead we will use ↓

JUL AUG

2018

22 FRIDAY JUNE

JUNE 2018

WK 25 (173-192)

(similar to selectors in CSS)

8 am Selecting Elements :-

9 am • getElementById
Returns the Element as an Object or ~~null~~ null (if not found).
e.g.
document.getElementById("id");
↳ It can be stored in a variable to ease and use its property any time/any where.
"id" can be stored in a string variable and passed without " ".

10 am

11 am

12 noon

1 pm

2 pm • getElementsByClassName
Returns the Elements as an HTML collection (similar to array) or empty collection (if not found)
e.g.
~~get~~ getElementsByClassName("className");
↳ (same ^)
So it returns all the HTML elements as a collection which has given class name.

3 pm

4 pm

5 pm

6 pm • getElementsByTagName
Returns the Elements as an HTML collection or empty collection (if not found)
e.g.
~~get~~ getElementsByTagName("tagName");
↳ (same ^)
So it returns all the HTML elements as a collection which has given tag name.
Here, tag name is not case sensitive (as in HTML)

2018

(↳ CSS selectors can not be used in all these)

2018 JULY

WK 25 (174-191)

23 SATURDAY JUNE

8 am • Query Selectors :- (most used)
Allows us to use any CSS selectors.
e.g.
document.querySelector("p"); // Selects first p element

9 am

10 am document.querySelector("#myId"); // Selects first element with id = myId.

11 am document.querySelector(".myClass"); // Selects first ele. with class = myClass.

12 noon ↳ (these only select first element)

1 pm To select ~~a~~ All elements ↴

2 pm e.g.
document.querySelectorAll("p"); // Selects all p elements.

3 pm ↳ (All CSS selectors can be used to select elements using Query Selectors).

4 pm (It gives collection of all p elements i.e. NodeList (length))

5 pm Using Properties & Methods :- (DOM manipulation Starts from Here)

6 pm • ~~inner~~ innerText
Shows the visible text contained in a node.

• textContent
Shows all the full text

• innerHTML
Shows the full markup.

JUL AUG

SUNDAY 24

2018

25 MONDAY JUNE

JUNE 2018

WK 26 (176-189)

Features	innerText	textContent	innerHTML
• Hidden content	No	Yes	Yes
• \n Content line-HTML doc based)	No	Yes	Yes
• HTML tags - All	No	No	Yes
• Next line (\n Web page doc based)	Yes	Yes	Yes
• Manipulation of content:	Yes	Yes	Yes
• Reads HTML tags when manipulating	No	No	Yes

e.g.
 Hello!
Hi,
I am Ritank.

JS:-

let div = document.querySelector("div");

div.innerText;
↳ 'Hi, I am Ritank.' (this is how it is shown on Web page)

div.textContent;
↳ '\n Hello! \n Hi, \n I am Ritank. \n'
(this shows hidden content also, and \n according to HTML file doc)

2018

2018 JULY

WK 26 (177-188)

26 TUESDAY JUNE

div.innerHTML;
↳ Whole HTML with tags

Manipulate :-
div.innerText = " Hi , I am Ritank.";
or div.textContent
↳ Hi , I am Ritank.
(This does not read HTML tag)

div.innerHTML = " Hi , I am Ritank.";
↳ **Hi**, I am Ritank.
(Reads HTML tags)

Using backticks to use variable :-

div.innerHTML = `${div.innerText} <br> Thankyou`;
↳ Hi, I am Ritank.
Thankyou

Manipulating Attributes :-

- obj.getAttribute("attr_name"); // getter
- obj.setAttribute("attr_name", "value_to_set"); // setter

In programming getters and setters are frequently used for object manipulation.

e.g. Hello! I am Ritank
JS:-
let para = document.querySelector("p");

2018

JUL AUG

27
WEDNESDAY
JUNE

JUNE 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20 21 22 23 24
25 26 27 28 29 30

WK 26 (178-187)

8 am para.getAttribute("id");
↳ 'paragraph'
9 am para.setAttribute("id", "line"); (CSS styling may disappear)
↳ (now id is changed to 'line')
10 am
para.getAttribute("class");
11 am ↳ Null (bec there is no class)
para.setAttribute("class", "hello");
12 noon ↳ (adds class attribute with value "hello")
1 pm Manipulating Style :- (with style attribute)
2 pm (hyphen based (-) CSS property is changed in JS to camelCase. e.g. background-color → backgroundColor)
3 pm
style Property (not much used)
4 pm ↳ obj.style (this property can only access inline properties, means CSS properties can not be accessed / shown)
5 pm
6 pm obj.style → will show all the properties of style associated with obj object i.e. HTML element(s), & can only show/access inline style in HTML, CSS can't be accessed.
This is why this property is not much used, and its better to use CSS for styling purpose.
e.g. obj.style.backgroundColor = "yellow";
And also it overrides CSS properties(s) (if any) because it manipulating inline styles which has highest preference.

2018

2018 JULY
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16 17 18 19 20 21 22
23 24 25 26 27 28 29 30 31

WK 26 (179-186)

28
THURSDAY
JUNE

- using classList :-
  obj.classList → shows all the classes given to the element(s)

Manipulating classes using classList ↴

- classlist.add()
- classList.add() → to add new classes
  e.g.
  obj.classList.add("abc");
  ↳ adds 'abc' class to element obj.
- classlist.remove() → to remove classes
  e.g.
  obj.classList.remove("abc");
  ↳ removes 'abc' class if there
- classList.contains() → to check if class exists
  e.g.
  obj.classList.contains("abc");
  ↳ true if 'abc' is there as class otherwise false is 'abc' class is not present.
- classList.toggle() → to toggle(on/off) between add & remove.
  e.g.
  obj.classList.toggle("abc");
  ↳ if 'abc' class is already then it is removed and returned false.
  else if 'abc' class is not present then it will be added and returned true.

obj.setAttribute could also be used to add/set class but it will remove all other classes. Hence, not used much to manipulate classes.

2018

JUL
AUG

29
FRIDAY
JUNE

JUNE 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20 21 22 23 24
25 26 27 28 29 30

WK 26 (180-185)

8 am

Navigation :-

9 am

- parent Element (only one parent possible)
  e.g.
  obj.parentElement;
  ↳ gives the parent of the element.

10 am

11 am

- children
  Multiple children can be possible of elements, hence it gives collection of children.
  e.g.
  obj.children;
  ↳ HTML Collection of children of obj
  obj.childrenElementCount;
  ↳ Given no. of children elements.

12 noon

1 pm

2 pm

3 pm

4 pm

- previousElementSibling / nextElementSibling
  It will give access to previous or next element sibling. (if there, if not there then NULL).
  e.g.
  obj.previousElementSibling
  ↳ gives access to previous element sibling of obj element.
  obj.nextElementSibling
  ↳ gives access to next element sibling of obj element.

5 pm

6 pm

obj.children[1].previousElementSibling;
(2nd children of obj)
gives access to 1st children of obj

2018

2018 JULY
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16 17 18 19 20 21 22
23 24 25 26 27 28 29 30 31

WK 26 (181-184)

30
SATURDAY
JUNE

8 am

Adding Elements :-

document.createElement()
It creates a new element, (but do not insert in doc)
e.g.
let newp = document.createElement("p");
↳ creates new paragraph element, but not inserted and most of the attributes will be empty.

9 am

10 am

11 am

12 noon

newp.innerText = "Hi, I am new p."
↳ ~~setting~~ setting inner text to to new paragraph.

1 pm

Inserting Elements :-

2 pm

- appendChild(element)
  It appends (adds at last) the element as child of given element at last
  e.g.
  let body = document.querySelector("body");
  body.appendChild(newp);
  ↳ appends new paragraph element inside body element at last
  Only variable can be passed not string.
  newp variable ~~can only~~ element can only be added once anywhere (not multiple times)
  ~~then~~

3 pm

4 pm

5 pm

6 pm

- append(element)
  It has multiple uses. It does all the work of appendChild. Also it can add ~~new~~ string/text directly. Hence it is ~~more~~ more used than appendChild.
  e.g.
  (same example as appendChild)

JUL
AUG

2018

NOTES

newp.append("That's all.")
↳ it will add ~~to~~ more text to newp.
"Hi, I am newp. That's all.

- prepend(element)
  Same as append, but it adds element / text in starting (first).
- insertAdjacent(where, element)
  where ↳ position

Positions :-

1. 'beforebegin' : Adds element before target element itself.
   1.e. <element> //here
   <target ele> </target ele>
   as sibling
2. 'afterend' : Adds element after target element itself.
   e.g. <target ele> </target ele>
   <element> //here

3. 'afterbegin' : Just inside target element, before its first child / text.
   as child
   e.g. <target ele> <element> //here </target ele>
   ↳ (starting)
4. 'beforeend' : Just inside target element, after last child / text.
   e.g. <target ele> //here <element> </target ele>
   ↳ (end)

Removing Elements :-

- removeChild(element) : (~~acts~~ as appendChild)
- remove(element) : (~~acts~~ as append) //More Used.

⇒ simply removes selected element or child element if there.

// Correction →

AUGUST 2018

M	T	W	T	F	S	S
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		

2018
JULY

MON	TUE	WED	THU	FRI	SAT	SUN
30	31					1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29

JUL
AUG

Remove Attribute :-
Format : • element.removeAttribute("attribute_name");

→ // correction Remove Elements

- element.removeChild(child_element);
- element.remove(); //More used

01
SUNDAY
JULY

JULY 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16 17 18 19 20 21 22
23 24 25 26 27 28 29 30 31

WK 26 (182-183)

8 am
9 am
10 am
11 am
12 noon
1 pm
2 pm
3 pm
4 pm
5 pm
6 pm

DOM Events :-
Events are signals that something has occurred. (user inputs / actions)
e.g. clicking buttons, taking inputs etc.

→ Inline : (Event in HTML only)
e.g. HTML ↴
```
<button onclick="console.log("button is clicked");">
    Click Me!
</button>
```

• onclick : attribute is used to do / trigger defined events on single mouse click when element is clicked.

Managing events through inline is not recommended because this is not efficient, can become bulky and reduce readability. Rather use JS.

• onclick : (when an element is clicked)
e.g. HTML ↴
```
<button> Click me 1 </button>
<button> Click me 2 </button>
<button> Click me 3 </button>
```

JS ↴
```
let btns = document.querySelectorAll("button");
                // Returns collection of all buttons.
function sayHello() {
    alert("Hello");
}
for (btn of btns) {
    btn.onclick =
```

2018

2018 AUGUST
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12
13 14 15 16 17 18 19 20 21 22 23 24 25 26
27 28 29 30 31

WK 27 (183-182)

02
MONDAY
JULY

8 am
9 am
10 am
11 am
12 noon
1 pm
2 pm
3 pm
4 pm
5 pm
6 pm

```
for (btn of btns) {
    btn.onclick = sayHello;
}
```
↓
no brackets () because then it will call function. But we just need to assign function. (And only single function can be executed)

Output ↴
[Click me 1] [Click me 2] [Click me 3]

When we click any button, there will be alert message "Hello".

• onmouseenter : (when mouse enters an element) Performs the action when mouse enters the area of the given element (s).

• dblclick : (when double clicked element)

There are several (more) events.] → Refer Mdn.

Event Listner :-
addEventListner.
Syntax / Format : element.addEventListner("event", callback)
Whenever particular event is triggered, event listner gets activated and performs given function / callback / task assigned.
Multiple tasks can be performed using addEventListner. This is advantage. And not limited only one action / task, like when using normal events.
Hence, whenever we want to use events, we will use it through addEventListner().

AUG

2018

03
TUESDAY
JULY

JULY 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16 17 18 19 20 21 22
23 24 25 26 27 28 29 30 31

WK 27 (184-181)

8 am
9 am
10 am
11 am
12 noon
1 pm
2 pm
3 pm
4 pm
5 pm
6 pm

e.g. in last code only :-

```
for (btn of btns) {
    btn.addEventListner("click", function() {
        alert("Hello");
    });
    btn.addEventListner("click", function() {
        alert("Hi");
    });
}
```

↳ Now both the alerts will pop up one by one.

'this' in Event listners :-

When 'this' is used in a callback of event handler of something, it refers to that something or the element on which we are applying event listner.

'this' can help to improve redundency of code.
e.g.
when we want to apply event listener for multiple elements, but callback is kind of same for everyone. Then 'this' comes in game.
e.g.

```
let btn = document.querySelector("button");
let p = "p";
let h1 = "h1";

function changeColor() {
    console.dir(this.innerText);
    this.style.backgroundColor = "blue";
}
```

removeEventListner :-

Format → element.removeEventListner("event", callback);

2018 AUGUST
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12
13 14 15 16 17 18 19 20 21 22 23 24 25 26
27 28 29 30 31

WK 27 (185-180)

04
WEDNESDAY
JULY

```
// now pass this function as callback for all elements
btn.addEventListner("click", changeColor);
p.add   "   ("click", changeColor);
h1.add  "   ("click", changeColor);
// for every callback 'this' will be the 'object/element' i.e. btn, p, h1
```

Keyboard Events :-

→ event parameter : in addEventListner, the callback can have a default parameter (which can be any name, prefered - 'event'). Which will give information/properties of the triggered event (s). It is actually an object itself which store all the properties of triggered event
e.g.

```
btn.addEventListner("dblclick", function(event) {
    console.log(event);
});
```

// Gives, MouseEvent object since double click is an event triggered by mouse, which gives all the information of double click.
This will help to track the triggered event (s).

- keydown : It is a keyboard Event, triggered when key is pressed
- keypress : It is a keybo
- keyup : It is a keyboard Event, triggered when key is released.
- keydown : It is

// More keyboard Events - Refer Mdn

AUG

2018

05
THURSDAY
JULY

JULY 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16 17 18 19 20 21 22
23 24 25 26 27 28 29 30 31

WK 27 (186-179)

8 am e.g. HTML ↴
```
<input placeholder="type something">
```
9 am JS ↴
```
let inp = document.querySelector("input");
inp.addEventListner("keydown", function(event){
    console.log(event);
    console.log(event.key); //Input properties
    console.log(event.code);
    consol.log("Key was pressed");
});
```
10 am
11 am
12 noon
1 pm
2 pm
3 pm
4 pm
5 pm
6 pm

↳ event → returns all the information when a key is pressed/entered
event.key → Name of key pressed
event.code → Symbol of key pressed.

e.g.

	key	code
a	~~Key~~ KeyA	a
b	KeyB	b
	Space	

	Key	code
a	"a"	"KeyA"
b	"b"	"KeyB"
	" "	"Space"
;	";"	"Semicolon" etc.

~~Form~~ Form Events :-
- submit : it is a ~~form~~ form event, triggered when we click submit or submit the form.

// For more refer mdn.

2018

2018 AUGUST
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12
13 14 15 16 17 18 19 20 21 22 23 24 25 26
27 28 29 30 31

WK 27 (187-178)

06
FRIDAY
JULY

→ event.preventDefault() :- It prevents the action to happen, after action is triggered.
e.g. we can prevent from submitting the form and move to another url using this.

8 am
9 am
10 am e.g. HTML ↴
```
<form action='/action'>
    <input placeholder="username">
    <button>Register</button>
</form>
```
11 am
12 noon
1 pm JS ↴
```
let form = document.querySelector("form");
form.addEventListner("submit", function(){
    event.preventDefault(); //this will prevent to enter another url, or submitting the form
    alert("You clicked Register");
});
```
2 pm
3 pm
4 pm

Extracting Data from Form :-
We can save data to a variable by two methods.
e.g.
5 pm
6 pm

HTML :↴
```
<~~form~~ form action="/action">
    <input placeholder="username" type="text" id="user">
    <input placeholder="password" type="password" id="pass">
    <button>Register</button>
</form>
```
JS :↴
```
let form = document.querySelector("form");
```

AUG

2018

07 SATURDAY JULY

WK 27 (188-177)

JULY 2018

M T W T F S S M T W T F S S
9 10 11 12 13 14 15 16 17 18 19 20 21 22
23 24 25 26 27 28 29 30 31

8 am

method 1;

form.addEventListner("submit", function(event) {
event.preventDefault();

10 am

let user = document.querySelector("#user");
let pass = document.querySelector("#pass");

11 am

alert(`Hi ${user.value}, your password is ${pass.value}`);

12 noon

});

1 pm

→ value : input values are stored in value property.

2 pm

Method 2: (Better)

3 pm

→ elements : It is a form element's property which stores collection of all the elements inside form.

4 pm

e.g.

form.elements;]
0: input #user
1: input #pass
2: button
etc.

5 pm

Now we can use this instead of previous code.

6 pm

e.g. inside addEventListner]

let user = this.element[0];
let pass = this.element[1];
(works same)

08 SUNDAY

In this we used form's element property to access the elements inside form.

2018

2018 AUGUST

M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12
13 14 15 16 17 18 19 20 21 22 23 24 25 26
27 28 29 30 31

WK 28 (190-175)

09 MONDAY JULY

8 am

change Event :-
The change event occurs when the value of an element has been changed (only works on <input>, <textarea> and <select> elements).
It tracks initial and final change only.

9 am

e.g.

10 am

let user = document.querySelector("#user");
user.addEventListner("change", function() {

11 am

console.log("input changed");
console.log("final value=", this.value);

12 noon

});

1 pm

If we change the input value then it will be triggered.

2 pm

input Event :-
The input event fires when the value of an <input>, <select>, or <textarea> element has been changed. It traks all small changes.

3 pm

e.g.

//

4 pm

user.addEventListner("input", function() {
// (same)

5 pm

});

If we even change even one/single letter, it will be triggered.

6 pm

Non characters keys will not (be) triggerked). the input event.

- mouseout : This event is fired at an element when a pointing device (usually a mouse pointer) is used to move the cursor so that it is no longer contained within the element or one on its children.

AUG

2018

10 TUESDAY JULY

WK 28 (191-174)

JULY 2018

8 am • Keypress : This event no longer in use / Is event.

9 am • scroll : This event fires when document view has been scrolled. Choose scrollable element or

10 am whole document itself.

11 am • load : This event is fired when the whole page has loaded, including all dependent

12 noon resources such as stylesheets, scripts, iframes & images. It will work with 'window' object.

1 pm

Event Bubbling :-

2 pm It happens when an event is triggered / fired, on given element which is inside another element.

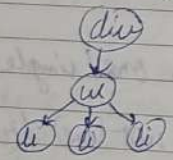
3 pm And this parent element has its own events, so that will be also fired due to event bubbling.

4 pm To stop this we can use :-

→ even.stopPropagation();

5 pm This will prevent from event bubbling and only triggers for child element only.

6 pm e.g.



JS :-

let div, ul, lis = document.querySelector("")

div.addEventListner("click", function() {
console.log("div was clicked");
}); // This is outer most parent, so stopPropogation is not needed.

2018

2018 AUGUST

11 WEDNESDAY JULY

WK 28 (192-173)

ul.addEventListner("click", function(event) {
8 am event.stopPropagation();
console.log("ul was clicked");
9 am });

10 am for(li of lis) {
li.addEventListner("click", function(event) {
11 am event.stopPropagation();
console.log("li was clicked");
12 noon }); // if stopPropogation was not used then by clicking li all 3 (li, ul & div) events would had triggered due to event bubbling.

1 pm

2 pm Event Delegation :-

When we add a new element through JS and

3 pm if there was event Listner for some type of element, the for new elements event Listner

4 pm will not apply. So, here comes Event Delegation.

Event delegation uses the concept of bubbling

5 pm to apply even listner on the new added element.

6 pm → event.target : It tells what exact (child) element was the cause to trigger the event. It has so many properties avilable.

→ event.target.nodeName : Returns exact element name that caused the event to trigger.

e.g. event.target.nodeName → "BUTTON"

→ event.target.parentElement: Return parent element of target ele.

So we first apply the event Listner to parent element, then through properties like event.target & even.target.nodeName we access new added element then apply the event. (// Check TODO Project)

2018

AUG

12 THURSDAY JULY

JULY 2018

WK 28 (193-172)

8 am Call Stack :-
It is a data structure which tracks the functions. It follows LIFO. (Last In First Out)
Stack stores function as LIFO manner.

10 am Breakpoints :- It is used to actual visualize the
11 am JS call stack and also used for debugging in browser.
Inspect → Sources → JS file → add Breakpoints → Debug.

12 noon

1 pm

2 pm Generally Programming Languages can be catagorized in two ways :-
3 pm Single Threaded : These are often straightforward and easy to work with. It execute one thing at a
4 pm time. It executes current line first before moving to next line
5 pm JavaScript is a Single Threaded Programming language.
Multi Threaded : These offer advantage of parallel
6 pm parallel execution, allowing tasks to be performed concurrently. This can lead to significant performance improvements especially in tasks that involve high computation, data processing or network operations.
e.g. C++, Java, Python, etc.

Javascript Synchronous & Asynchronous Nature :-
Synchronous - It means to be in sequence i.e. every statement of the code gets executed one by one.
So, basically a statement has to wait for the earlier statement to be executed.

2018

2018 AUGUST

13 FRIDAY JULY

WK 28 (194-171)

8 am Asynchronous JS - It allows the program to be executed
9 am immediately whereas synchronous code will block further execution of the remaining code until
10 am it finishes the current one. This may not look like big problem but when we see bigger picture we realize that it may lead to delaying the User Interface.
11 am To tackle problem like this we have some concepts/functions
12 noon /methods in JS. e.g. setTimeout, setInterval, callbacks (Hell), Promise, etc.

1 pm JS is single threaded then how setTimeout works?
JS is single threaded but browser is not.
2 pm Browsers are capable of multithreading (since it is coded in languages like C++). Browser will set Timeout
3 pm code until time hits and add it to JS call stack and finally gets executed.
4 pm Javascript cannot wait some code to execute, this responsibility is given to browser, browser
5 pm waits and add the code to call stack at right time.

6 pm Callback Hell :-
It is essentially nested callbacks stacked below one another forming pyramid structure. Every callback stacked below one another depends/waits for the previous callback, thereby a pyramid structure that affects the readability and maintainability of the code.
e.g.
HTML:
<h1> Hello World! </h1>

AUG

2018

14
SATURDAY
JULY

WK 28 (195-170)

JULY 2018

JS:

```
h1 = document.querySelector("h1");

function changeColor(color, delay, nextColor) {
    setTimeout(() => {
        h1.style.color = color;
        if (nextColor) nextColor();
    }, delay);
}

changeColor("red", 1000, () => {
    changeColor("orange", 1000, () => {
        changeColor("yellow", 1000, () => {
            changeColor("blue", 1000);
        });
    });
});

// Nested callbacks → Callback Hell
```

This can be confusing for larger code. And also this exists in industry.

Q. Upload data on the basis of internet speed.
Assume internet speed using Math.random(). [0, 10]
if internet spee > 4 → Upload data & move to next
else Failure. Stop.

15 SUNDAY

e.g. Suppose we have to upload "Neha", "Aman", "Raj" but in sequence and no compromise.
Cases: 1. Neha Uploaded ✓ Continue
Aman Weak connection ✗ Stop

2018

2018 AUGUST

16
MONDAY
JULY

WK 29 (197-168)

2. Neha ✓ 3. Neha ✓ 4. Neha ✗
Aman ✓ Aman ✓
Raj ✗ Raj ✓

→ VSCode → JS

Using callbacks we will step into callback hell.
For larger data it will be difficult to track and it will reduce readability and asynchronous in nature

Promises :—

The Promise object represents eventual completion (or failure) of an asynchronous operation and its resulting value.

completion → success / resolve] Mendatory arguments when
failed → failure / reject] return

→ These two are two possible [PromiseResult].

[PromiseState] → fulfilled
→ pending
→ rejected

Promise Method ; then() & catch()

- then() - When promise is fulfilled. Multiple then() can be implemented over single catch(). i.e. Promise Chaining
- catch() - When promise is rejected.

then(result) → then can take argument which can be used to see the result info. or fulfilled promise info.

catch(error) → Similarly it will show error info. if promise rejected.

2018

AUG

17
TUESDAY
JULY

JULY 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16 17 18 19 20 21 22
23 24 25 26 27 28 29 30 31

WK 29 (198-167)

8 am
9 am
10 am
11 am
12 noon
1 pm
2 pm
3 pm
4 pm
5 pm
6 pm

Q. Last questions but this time with Promise Chaining.

```
let arr = [];
function saveToDb(data) {
    return new Promise ((resolve, reject) => {
        let internetSpeed = Math.floor(Math.random()*10);
        if (internetSpeed > 4){
            arr.push(data);
            resolve("Success: Data Saved");
        }
        else {
            reject("Failure: Weak Connection");
        }
    })
} // Return Promise Object.

saveToDb("Neha")                    // Function call
    .then((result) => {
        console.log("Data saved");
        console.log("Result-", result);
        return saveToDb("Aman");
    })
    .then((result) => {
        //
        //
        return saveToDb("Raj");
    })
    .catch((error) => {
        console.log("Promise was rejected");
        console.log("Error-", error);
    });
```

18

2018 AUGUST
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12
13 14 15 16 17 18 19 20 21 22 23 24 25 26
27 28 29 30 31

WK 29 (199-166)

18
WEDNESDAY
JULY

8 am
9 am
10 am
11 am
12 noon
1 pm
2 pm
3 pm
4 pm
5 pm
6 pm

↳ This does the same work as last question which used callback hell. But now with Promise Chaining it is much readable and smooth.

```
console.log(saveToDb("Neha"));
```
→ returns / prints Promise Object.

all then() executes sequentially, hence if any of the then() is rejected, then it will directly jump to catch() function / statement and ignores other then()'s.

Async Functions :-

- async keyword :
  Creates an Async Function. (Object)
  By default async functions returns promise. Then we can apply Promise methods.

e.g.

```
async function greet() {
    return "Hello"; // [Promise Result] : "Hello"
} // if not returned then [Promise Result] is undefined
// now using then() & catch() ->
greet()
    .then((result) => {
        console.log("Promise was resolved");
        console.log("Result-", result);
    })
    .catch((error) => {
        console.log("Promise Rejected. Error :-", error);
    });
// Since there is no error inside async function then() statement will execute.
```

AUG

2018

19
THURSDAY
JULY

WK 29 (200-165)

JULY 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16 17 18 19 20 21 22
23 24 25 26 27 28 29 30 31

8 am → throw : this key is used to throw error
e.g. in last code
async function greet () {
throw "404 page not found";
return "Hello";
}
9 am
10 am
11 am Now catch() function will execute, and error will be : 404 page not found. Or if there is any error inside async function then catch() will execute.
12 noon
1 pm
async with arrow function :
2 pm e.g. let demo = async () => {
return 1;
}
3 pm demo(); → Returns Promise Object.
4 pm
• await keyword :
5 pm pauses the execution of its surrounding async function until the promise is settled (resolved or rejected). await keyword can only be used inside async function.
6 pm This can be used to even minimize the code of Promise chaining to some extent.
e.g. In color change question :
function ChangeColor (color, delay) {
return new Promise ((resolve, reject) => {
setTimeout (() => {
h1.style.color = color;
console.log ("Color Changed!");
resolve ("Color Changed!");
}, delay);
});
}

2018

2018 AUGUST
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12
13 14 15 16 17 18 19 20 21 22 23 24 25 26
27 28 29 30 31

WK 29 (201-164)

20
FRIDAY
JULY

async function colors () {
8 am await changeColor ("red", 1000);
await changeColor ("blue", 1000);
9 am await changeColor ("green", 1000);
}
10 am // Every await waits for changeColor function to execute before moving to next changeColor.
11 am
→ Handling rejections with Await :
12 noon Just use try() and catch() inside async function.
e.g.
1 pm async function color() {
try {
2 pm await changeColor (" ");
await changeColor (" ");
3 pm await changeColor (" ");
}
4 pm catch (err) {
console.log (err);
5 pm }
}
6 pm If any statement of await gets rejected, the catch will execute, displaying the error. And if there is any statement after catch, then that will easily execute.

AUG

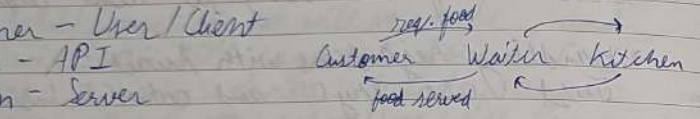
2018

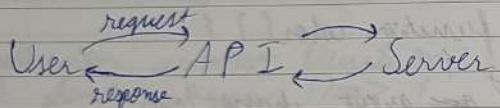
21 SATURDAY JULY

JULY 2018

WK 29 (202-163)

API :- Application Programming Interface
It is a way for two or more computer programs to communicate with each other, in general.
We will be dealing here with Web APIs, which works with http
e.g.
Customer - User / Client
Waiter - API
Kitchen - Server





API → URL / Link / endpoint

Some Random APIs (Free)
https://catfact.ninja/fact → Send random cat facts
https://www.boredapi.com/api/activity → Sends an activity to do when bored.
https://dog.ceo/api/breeds/image/random → Sends cut dog pictures link.

Some type of APIs (URL) could be completely free. ✗Key ✗Paid
another could be free but with key. ✓Key ✗Paid
or could be partial free, after some time it may be paid. So there are Paid APIs also. ✓Key ✓Paid.

There are other APIs like twitter API, instagram API, google API, etc. There is documentation available with every API, so it will be a good practice to go through documentation first.

22 SUNDAY

2018

2018 AUGUST

WK 30 (204-161)

23 MONDAY JULY

JSON :- APIs Return data in JSON format.
JavaScript Object Notation (www.json.org)
As the name suggest, but JSON is not only for JavaScript, it works for other programming languages like Python, django.
Before JSON, APIs used to return data in XML format. But now mostly JSON are in use.

XML was similar to HTML format.
JSON is similar to JS object.
Unlike JS object, JSON keys are in string format.
JSON value / data types :
object, array, string, number, "true", "false", "null".
"undefined" is not in JSON.

JSON validator can be used to validate JSON code.

Accessing Data from JSON :
Generally data returned from API i.e. in JSON format is a complete string.
To fetch there are some functions available.

- JSON.parse(data) - Method
  To parse a string into JS object.
  e.g.
  let JSONRes = '{"name":"Rishank", "age":22}'; // string
  let validRes = JSON.parse(JSONRes);
  // Now JSONRes is converted to object validRes.
- JSON.stringify(json) - Method
  To parse a JS object data into JSON.
  Just reverse of previous method.
  Here we pass a JS object to convert into JSON

2018

24 TUESDAY JULY

JULY 2018

WK 30 (205-160)

8 am Testing API requests :-
2 famous tools :
- Hoppscotch (will be using)] Both have similar interface.
- Postman

9 am

10 am

Ajax :-
11 am Asynchronous JavaScript and XML
This is the process through which we call API and in response we get data. This whole process is asynchronous in nature.

12 noon Ajax should be Ajaj because now JSON Response are in use ~~but~~ than XML, but this is an old name, ~~and~~ no one changed it yet.

1 pm

2 pm

Http Verbs :- (on Hoppscotch)

3 pm e.g.
- GET
- POST
- PUT
- PATCH
- DELETE

4 pm etc.

5 pm We will be only dealing with GET for now.
Others we will be dealing when we start server side coding or building APIs.

6 pm Status Codes :-
When response comes back from API, there are several status codes which tells the status of the request / response.

18

2018 AUGUST

WK 30 (206-159)

25 WEDNESDAY JULY

8 am e.g.
- 200 - Ok
- 404 - Not Found
- 400 - Bad Request
- 500 - Internal Server Error.

etc. Refer Status Code Mdn.

9 am

10 am

11 am Additional Information in URLs (API endpoint) :-
Query Strings :

12 noon e.g. https://www.google.com/search?q=harry+potter
(q = key, harry+potter = value)

1 pm

(URL) ~~key~~ ? name = ritank & marks = 95 & age = 22
(name = key1, ritank = value1, marks = key2, 95 = value2, age = 22 → This will be ignored if not valid key-value pair)

2 pm (key is constant, value can be variable)

3 pm url/
- :id
- :num
- :q
- {{id}}
- <id>

4 pm There all are variables, should be replaced with valid id/value.

5 pm

6 pm Http headers :-
header, value
It is basically used to supply additional information for both request and response.
Headers can send meta data (data about data)
Headers can be altered by providing header name and value with it on Hoppscotch.

2018

AUG

26 THURSDAY JULY

JULY 2018

WK 30 (207-158)

First Request :-
using Fetch : fetch(url)
Before fetch XMLHTTPRequest obj was in use, but outdated now.

fetch(url); → returns Promise, hence we can apply Promise methods / chaining

e.g. let url = "https://catfact.ninja/fact";
fetch(url)
.then((res) => {
    console.log(res); // This returns the Response object if url request was correct
// console.log(res.json()); // This returns // This again returns the Promise, through we can access readable data
    return res.json();
})
.then((data) => {
    cond.log(data); // This returns readable JSON (object)
    console.log(data.fact); // Prints the data of 'fact' key of JSON
    return fetch(url); // Requesting again and returning
})
.then((res) => {
    return res.json();
})
.then((data) => {
    console.log(data.fact);
})

2 step process

2018

2018 AUGUST

WK 30 (208-157)

27 FRIDAY JULY

.catch((err) => {
    console.log(err);
}); // If there is any error in any then(), catch will execute, printing the exact error.

This whole process is asynchronous in nature.

using Fetch with async & await :-
e.g.
async function getFacts() {
    try {
        let res = await fetch(url)
        let data = await res.json();
        console.log(data.fact);
            // Since API call is asynchronous, hence using await
        let res2 = await fetch(url); // Second API call
        let data2 = await res2.json();
        console.log(data2.fact);
    }
    catch(err) {
        console.log(err);
    }
} // Using try & catch so that further execution won't stop if there is any error.

Used await (or then) for both fetch(url) and res.json(), because even after fetch(url) getting response from fetch(url) which returns some header, but res.json() can still take time or we do not get 100% response with fetch(url) instantly.

2018

AUG

Free API has limits for API calls.

28 SATURDAY JULY

JULY 2018

WK 30 (209-156)

8 am Axios :-
Library to make HTTP requests.
9 am Internally it uses fetch() only, but it is more compact and better to send HTTP request or API call.
10 am
(Github → axios → copy cdn link to HTML)
11 am
Why axios? fetch() response was in Redable Stream but not exact data, so we had to parse.
12 noon But in ~~axios~~ axios we get exact data.
1 pm e.g. Random Cat Fact
HTML:
2 pm <h1> Get Random Cat Fact </h1>
<button> show cat fact </button>
3 pm
JS:
4 pm async function getFacts() {
try {
5 pm let res = ~~await~~ await axios.get(url);
return res.data.fact;
6 pm }
catch(err) {
console.log(err);
}
29 SUNDAY }
let btn = document.querySelector("button");
btn.addEventListner("click", async () => {
let fact = await ~~get~~ getFacts();
let p = document.querySelector("p");
p.innerText = fact;
2018 }); // using async so use await.

2018 AUGUST

WK 31 (211-154)

30 MONDAY JULY

Sending Headers with Axios :-
8 am e.g.
const config = { headers: { Accept: "application/json" } };
9 am let res = await axios.get(url, config);
// inside async function.
10 am console.log(res.data); // Returns data in JSON format.
11 am Axios ~~Update~~ : Updating Query Strings :-
e.g.
12 noon let url = "http://universities.hipolabs.com/search
?name=";
1 pm let country = "india";
// in async function
2 pm ~~let~~ let res = await axios.get(url+country);
console.log(res);
3 pm // returns all universities in India
4 pm Q. Search Universities by country/state; → JS
5 pm
6 pm

2018

one
commits)
resets to previous
commit, now only
modified left.

y commits)
hash with comments
(new to old)
resets to selected
, modified left.
> → resets and
he changes gone
modified

res and visibility
repository.
story.

10 11 12 13 14 15 16 17 18 19 20 21 22 23
24 25 26 27 28 29 30
WK 32 (223-142)

11
SATURDAY
AUGUST

OOPS Concepts :-

8 am OOP is programming paradigm / model based on the concept of classes & objects, which can contain data and code: data in form
9 am of fields & code in form of procedures. A common feature of objects is that methods are attached to them and can
10 am access and modify the object's ~~code~~ data fields.

Topics :-

11 am • prototypes • New Operator • Constructors • classes
• Keywords (extends, super)

12 noon

Object Prototypes :-

1 pm Prototypes are the ~~not~~ mechanism by which JavaScript objects inherit features from one another.
2 pm It is ~~it~~ a like a single template object that all objects inherit methods and properties from without
3 pm having their own copy.
Every object in JavaScript has a built-in property
4 pm which is called its prototype. The prototype is itself an object, so the prototype will have its own properties,
5 pm making what's called a prototype chain. The chain ends when we reach a prototype that has 'null' for
6 pm its own property.

e.g. The array which is an object in JavaScript. Array has its own ~~prototypes~~ prototype which includes methods and properties e.g. .pop(), .push(), etc. Hence every new array created will inherit array's prototype. And this prototype does not take extra space for every new array. It just take single space. It works on the basis of pointer. Prototype which is object itself is stored in some memory location, array which array can access by pointing internally every

SUNDAY 12

2018

8 am
- arr.__proto__ → using this we can access the prototype defined for the array/object/etc. by referencing.
9 am
The prototype which store methods and properties can be altered by using this also. ~~(But be~~ Changing built-in properties is also possible but not recommended.
10 am
e.g. let arr = [1, 2, 3];
11 am
arr.__proto__.push = (n) => { console.log("Hello"); };
↳ push array push methode will be altered/changed. This will apply to all the arrays.
12 noon
- Array.prototype → to access actual prototype object of/for an array directly. This shows all the built-in properties/methods for an array.
1 pm
- String.prototype → // same for string.
2 pm
→ defining function(s)/method(s) for an array :
3 pm
e.g. let arr1 = [1, 2, 5];
arr1.sayHello = () => { console.log("Hello! I am in arr"); };
4 pm
This sayHello() function/method is not part of the prototype, hence if we want to use this function for other arrays we will have to add it again and again. This will also take extra space.
5 pm
e.g let arr2 = [4, 5, 6];
arr2.sayHello = () => //.
6 pm
arr1.sayHello === arr2.sayHello
↳ False → because both have diff. memory location
arr1.push === arr2.push
↳ True → 'push' method is in prototype Hence same memory location.

Disadvantage of Factory Functions to create Objects:-
8 am e.g. Creating a function to store every person details.
function PersonMaker(name, age) {
9 am const person = {
name: name,
10 am age: age,
talk() {
11 am console.log(`Hi, my name is ${this.name}`);
},
12 noon };
return person;
}
1 pm let p1 = PersonMaker("adam", 25);
2 pm let p2 = PersonMaker("eve", 25);
3 pm // The disadvantage here is every person will have method talk(), but stored in different memory location for ever person. Which will be very ineffieient way.
4 pm p1.talk === p2.talk
↳ false → stored in different memory location
5 pm which means every person will have a copy of talk() method in diff. memory location.
6 pm

New Operator :-
The new operator lets developers create an instance of a user-defined object type or of one of the built-in object types that has a constructor function.
Constructors are/is a special function that creates and initializes an object instance of a class.

instance(s) is/are the objected created using constructor.

15
WEDNESDAY
AUGUST

AUGUST 2018

WK 33 (227-138)

In Javascript, constructor gets called when an object is created using new keyword.
The purpose of a constructor is to create a new object and set values for existing object properties.
Constructor does'nt return anything & start with ~~capital~~ capital letter, just for convention. (developers rule)
e.g.

```
// Constructor
function Person (name, age) {
    this.name = name;
    this.age = age;
}

// Prototype
Person.prototype.talk = function() {
    console.log(`Hi, my name is ${this.name}`);
} // do not use arrow function ~~here~~ here.

// Instances using new keyword
let p1 = new Person("adam", 25);
let p2 = new Person("eve", 25);

p1.talk === p2.talk
↳ true → Prototype
```

When a function is called with 'new' keyword, the function will be used as constructor. 'new' will do following things :-
1. Create a blank, plain JS object. It's called 'newInstance'.
2. Points 'newInstance's [[Prototype]] to constructor function prototype property, if there is any.
e.g p1 can access talk() property as it's prototype property of Person constructor.

2018

2018 SEPTEMBER

WK 33 (228-137)

16
THURSDAY
AUGUST

3. Executes the constructor function with the given arguments by the 'newInstance', binding 'newInstance's arguments ~~with~~ with constructor's 'this' properties accordingly.
4. The constructor function (returns) the 'newInstance' finally.

Understanding with 'this' :-

```
function Person (name, age) {
    this.name = name;
    this.age = age;
    console.log(this);
}
// calling Person() normally
Person("adam", 25);
↳ Prints 'window' object
// calling Person() using 'new'
let p1 = new Person("adam", 25);
↳ Prints the 'new Instance' created that is p1.
i.e. Person {name: 'adam', age: 25}
        ~~no~~ age : 25
        name : "adam"
        [[Prototype]] : Object
```

~~Hurt~~ Hence, this is level up and better than factory function.

Classes :- (level up / better)
Classes are template for creating objects.
The constructor method is special method ~~to~~ of a class for creating and initializing an object instance of that class. Should start ~~from~~ with capital letter for convention.
e.g.

2018

17
FRIDAY
AUGUST

AUGUST 2018
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12
13 14 15 16 17 18 19 20 21 22 23 24 25 26
27 28 29 30 31

WK 33 (229-136)

8 am
```
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }
    talk() {
        console.log(`Hi, my name is ${this.name}`);
    }
}

// This is working same as previous but it has
better/more readable syntax. talk() is part
of prototype.

let p1 = new Person("adam", 25);
let p2 = new Person("eve", 25);

p1.talk === p2.talk
  ↳ true (talk() is prototype)
```

Inheritance :-
It is a mechanism that allows us to create new classes on the basis of already existing classes.
e.g. parent class (base class)
↓ (inherit)
child class

- extends
- super

] these keywords used for inheritance

e.g.
```
class Mammal { // base class / parent
    constructor(name) {
        this.name = name;
        this.type = "warm-blooded";
    }
    eat() {
        console.log("I am eating");
    }
}
class Dog extends Mammal { // child
    constructor(name) {
        super(name);
    }
    bark() {
        console.log("woff..");
    }
    eat() {
        console.log("dog is eating");
    } // Overriding / Overruled parent prototype eat()
}
class Cat extends Mammal { // child
    constructor(name) {
        super(name)
    }
    meow() {
        console.log("meow..");
    }
}
let dog1 = new Dog("Bolt");
let cat1 = new Cat("Coco");
```

2018

2018 SEPTEMBER
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9
10 11 12 13 14 15 16 17 18 19 20 21 22 23
24 25 26 27 28 29 30

WK 33 (230-135)

18
SATURDAY
AUGUST

JS OOP Summary Sheet :-
SUNDAY 19

Q1. What is Object Oriented Programming (OOP)?
Ans. OOP is programming paradigm in computer science that relies on the concept of classes and objects. It is used to structure a program into simple, reusable pieces of code blueprints (usually called classes), which are used to create individual instances of objects.

2018

20 MONDAY AUGUST

AUGUST 2018

WK 34 (232-133)

Q2. What are benefits of OOP in JS?
Ans. Some benefits are :-
- Improved code organization (structure of code)
- Reusability of code.
- Better maintainability of code
- Closeness to real-world objects.

Q3. What is diff b/w objects & class in JS?
Ans. Objects are JS stand alone entity, with properties, method and a type. It can can be created directly from functions or through constructor functions.
Class in JS acts as a blueprint for creating objects.

Q4. What is constructor function in JS?
Ans. Constructor function is special function that is used to create and initialize objects in JS. When a new object is created using a constructor function, it is automatically assigned a set of properties and methods that are defined within the function or associated with function i.e. Prototypes.

Q5. What is prototype chain in Javascript?
Ans. Every object in JS has built-in property, which is its prototype. Prototype is itself an object. So the prototype will have its own property, making what's called a prototype chain. The chain ends when we reach a prototype that has null for its own prototype.

2018

2018 SEPTEMBER

21 TUESDAY AUGUST

WK 34 (233-132)

Q6. What is the difference between a constructor and class in JS?
Ans. A constructor is a function that creates object, while class is a blueprint for creating objects. Classes define the framework, whereas, constructor actually creates the objects & initializes them. (In JS, classes are syntatic sugar over constructor functions)

Q7. Why is the "new" keyword used in Javascript?
Ans. The 'new' keyword is used to create an instance of an object. When used with a constructor function, it creates a new object and sets the constructor function's 'this' keyword to point the new object.

Q8. What is Inheritance in OOP?
Ans. Inheritance in OOP is defined as the ability of a class to derive properties & characteristics from another class while having its own properties as well.

Q9. What is the "super" keyword in JS?
Ans. The super keyword in JS acts a reference variable to parent class. It is mainly used when we want to access a variable, method or constructor from parent/base class to/for the child/derived class.

Q10. What is the "extends" keyword in JS?
Ans. The extend keyword is used in class declarations or class expressions to create a class that is a child of another class/parent class. Basically extends keyword is used for inheritance in JS.

2018